

DirectML Memory Optimization & Cache Management Guide

Comprehensive Architectural Analysis & Mitigation Strategies for PyTorch DirectML Backend

Target Framework: PyTorch
(torch_directml)

Underlying API: DirectX 12 / DirectML

Document Version: 2.0 (Deep Dive)

1. Root Cause & Architectural Overview

Unlike the standard NVIDIA CUDA backend (which exposes explicit memory management primitives such as `torch.cuda.empty_cache()` and `torch.cuda.memory_allocated()`), the **Microsoft DirectML extension** (`torch_directml`) **lacks a native cache clearing mechanism**. Understanding why requires examining the interaction between PyTorch's custom extension architecture and DirectX 12 graphics memory heaps.

Architectural Limitation

`torch_directml` connects to PyTorch through the custom `PrivateUse1` device backend mechanism. PyTorch's primary C++ caching allocator (`CUDACachingAllocator`) is designed specifically for CUDA/ROCm execution contexts. DirectML tensors wrap underlying DirectML buffers in opaque handles (`OpaqueTensorImpl`), rendering them invisible to standard PyTorch internal memory trackers.

1.1 DirectX 12 Heap Management Behavior

DirectML relies internally on the `dml_heap_allocator` built on top of DirectX 12 (DX12) memory management primitives. When a tensor is freed in Python, the DX12 allocator does *not* immediately yield the underlying physical VRAM back to the operating system or graphics driver. Instead:

- **Suballocated Heap Retention:** DX12 allocates memory in coarse, multi-megabyte heap blocks. Small tensor allocations are suballocated within these heaps. Even if individual tensors are destroyed, the parent heap remains active until all suballocations are freed.
- **Deferred Garbage Collection:** To minimize allocation latency during continuous GPU operations, the DirectML runtime defers memory de-allocation and reuses committed memory heaps across iterations.
- **Driver-Level Caching:** GPU driver vendors (AMD, Intel, NVIDIA) implement aggressive caching strategies on top of DirectX 12 resources. Memory marked as freed by DirectML may remain reserved in system RAM/VRAM according to driver heuristics.

2. Detailed Mitigation Strategies

Strategy 1: Python Reference Cleaning & Active Garbage Collection

Because DirectML cannot force-empty its internal pool on demand, developers must ensure that Python's reference counting system drops all handles to dead tensors immediately. If a tensor variable persists in the Python scope or inside an autograd computation graph, the underlying DX12 buffer will remain locked.

```

import gc
import torch
import torch_directml

# Initialize DirectML execution target
device = torch_directml.device()

def train_epoch(model, train_loader, optimizer, criterion):
    model.train()
    running_loss = 0.0

    for batch_idx, (data, target) in enumerate(train_loader):
        # Move tensors to DirectML device
        data = data.to(device)
        target = target.to(device)

        optimizer.zero_grad(set_to_none=True) # Frees memory stored in .grad attributes

        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        # CRITICAL: Extract scalar item to detach autograd history from accumulators
        running_loss += loss.detach().item()

        # Manually release local references to prevent scope leak
        del data, target, output, loss

    # Force Python's cyclic garbage collector to purge unreferenced objects
    gc.collect()
    return running_loss / len(train_loader)

```

Pro Tip: `optimizer.zero_grad(set_to_none=True)`

Setting gradients to `None` instead of zeroing them out completely deletes the gradient tensor allocations from VRAM, saving substantial memory between backward passes.

Strategy 2: Preventing Autograd Graph Memory Leaks

The single most common cause of unbounded VRAM backlog accumulation during training is implicitly retaining PyTorch's execution graph. In PyTorch, tensor operations construct an acyclic graph required for backpropagation.

If you execute `total_loss += loss` during a logging loop, the variable `total_loss` maintains a pointer to `loss`, which holds a pointer to the entire backward computational tree for every batch across the entire epoch.

```

# INCORRECT (Causes catastrophic memory leak):
total_loss += loss # Retains full autograd graph in VRAM

# CORRECT:

```

```
total_loss += loss.detach().item() # Extracts raw scalar float; discards computational graph
```

For validation, testing, or inference routines where backpropagation is not required, wrap the execution block with `torch.no_grad()` or `torch.inference_mode()` to disable graph construction entirely:

```
with torch.no_grad():
    for data, target in val_loader:
        data = data.to(device)
        preds = model(data)
        # Process validation predictions...
```

3. Subprocess Isolation (Guaranteed VRAM Flush)

When executing long training runs with frequent evaluation steps, memory fragmentations inside the DirectX 12 heap can accumulate to a point where ‘DML allocator out of memory!’ errors occur despite low active tensor usage.

Because the OS reclaims DirectX 12 hardware contexts only when a process terminates, the single most reliable way to achieve a complete, 100% VRAM reset (functionally identical to restarting the script) is offloading inference or evaluation tasks into a dedicated worker `subprocess`.

3.1 Subprocess Implementation Pattern

```
import torch
import torch_directml
import torch.multiprocessing as mp

def _isolated_eval_worker(model_weights_path, test_data_path, return_dict):
    device = torch_directml.device()
    model = BuildModel().to(device)
    model.load_state_dict(torch.load(model_weights_path))
    model.eval()

    test_loader = load_data(test_data_path)
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()

    return_dict['accuracy'] = correct / len(test_loader.dataset)

def run_isolated_evaluation(model_weights_path, test_data_path):
    mp.set_start_method('spawn', force=True)
    manager = mp.Manager()
    return_dict = manager.dict()

    p = mp.Process(target=_isolated_eval_worker, args=(model_weights_path, test_data_path,
return_dict))
```

```

p.start()
p.join() # Wait for process to terminate

return return_dict['accuracy']

```

4. Summary Comparison Matrix

TECHNIQUE	VRAM CLEARANCE LEVEL	IMPLEMENTATION COMPLEXITY	PERFORMANCE OVERHEAD	RECOMMENDED USE CASE
<code>torch.cuda.empty_cache()</code>	0% (Incompatible)	N/A	None	Unsupported on DirectML
<code>del tensor</code> + <code>gc.collect()</code>	30% – 70% (Frees dead tensors)	Low	Very Low (<1ms)	Every training iteration / epoch end
<code>optimizer.zero_grad(set_to_none=True)</code>	20% – 40% (Reclaims grads)	Low	Negligible	Before every backward pass
<code>loss.detach().item()</code>	Prevents Leak	Low	None	Metric logging & accumulators
Subprocess Worker (<code>mp.Process</code>)	100% (Hard OS Reset)	Medium	Process Spawn (~200ms)	Validation runs, evaluation loops, checkpoints